

“ONLINE LEARNING MANAGEMENT SYSTEM (LMS)”

A Project Report Submitted in partial fulfillment of the requirements for the award of the
degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

VENKAT SIDDHARTH REDDY MOKU 2520090026

SAI SAKETH V 2520090055

DHANUSH REDDY 2520090028

Under the Esteemed Guidance of

Divya Priya

Assistant Professor, Department of Computer Science and Engineering

Course: DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND
DEVELOPMENT (25CS1302E)

Academic Year: Y25, 2026–2027 | Trimester 4 | Team 27 | Section 11

K L (Deemed to be) University

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Declaration

The Project Report entitled “**ONLINE LEARNING MANAGEMENT SYSTEM (LMS)**” is a record of bonafide work of **Venkat Siddharth Reddy Moku - 2520090026, Sai Saketh V - 2520090055, Dhanush Reddy - 2520090028** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 11, Team 27, during 2026–2027, Trimester 4, at K L (Deemed to be) University. The results embodied in this report have not been copied from any other student, department, or institution.

VENKAT SIDDHARTH REDDY MOKU - 2520090026

SAI SAKETH V - 2520090055

DHANUSH REDDY - 2520090028

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report entitled “**ONLINE LEARNING MANAGEMENT SYSTEM (LMS)**” is a bonafide work done and submitted by **Venkat Siddharth Reddy Moku - 2520090026, Sai Saketh V - 2520090055, Dhanush Reddy - 2520090028** in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 11, Team 27, at K L (Deemed to be) University during 2026–2027, Trimester 4.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The completion of this project would not have been possible without the timely help and guidance of many people. We wish to express our sincere thanks to everyone who assisted us in one way or another during the course of this work.

We express our deepest gratitude to our Course Coordinator, Dr. K. V. Ramanathan, and to our guide, Divya Priya, for their support, encouragement, and guidance throughout the project.

We thank Dr. Hari Kiran Vege, Head of the Department, for providing the facilities and resources that allowed us to complete this project-based course.

We also thank the teaching and non-teaching staff who assisted us directly or indirectly, and our parents, friends, and classmates for their help and cooperation during our work.

VENKAT SIDDHARTH REDDY MOKU - 2520090026

SAI SAKETH V - 2520090055

DHANUSH REDDY - 2520090028

ABSTRACT

The Online Learning Management System (LMS) is a centralized, database-backed web application developed for the course Database Systems Engineering and Distributed Backend Development (25CS1302E). Traditional educational tools suffer from fragmented lecture sharing, lack of real-time visibility into student attendance and progress, manual reconciliation of assessments, and absent verification mechanisms. The proposed system resolves these challenges through a three-tier architecture comprising a modern React.js frontend, a scalable Node.js and Express.js REST API layer, and a structured MySQL relational database. The platform establishes strict Role-Based Access Control (RBAC) across three principal roles: Student, Instructor, and Administrator. Key subsystems include structured course curriculum authoring, high-performance cloud media streaming via AWS S3 pre-signed URLs, an automated continuous assessment evaluation engine, real-time course discussion channels powered by Socket.io, and a weighted LTPS (Lecture-Tutorial-Practical-Skilling) attendance calculation ledger ensuring 75% end-semester exam eligibility. Complete data normalization up to BCNF guarantees transactional integrity and eliminates update anomalies across multi-entity relationships.

At the Review-2 stage the project was approximately 40% complete. The abstract and scope, the architecture and technology stack, the database schema, the frontend page designs, and the JWT authentication flow had been finalized or designed. Connecting the frontend to live REST APIs, the video upload pipeline, the quiz engine with auto-grading, the Socket.io discussion forum, downloadable PDF certificates, and end-to-end testing remain as planned work. This report documents the design and status of the system and keeps designed, planned, and implemented functionality distinct.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION.....	1
1.1 Background of the Project.....	1
1.2 Problem Statement.....	1
1.3 Scope of the Project.....	2
1.4 Objectives.....	3
1.5 Importance of the Application.....	3
1.6 Target Users / Audience.....	3
CHAPTER 2: SYSTEM REQUIREMENTS.....	5
2.1 Hardware Requirements.....	5
2.2 Software Requirements.....	5
2.3 Development Tools and Frameworks.....	6
2.4 Functional and Non-functional Requirements.....	6
CHAPTER 3: TECHNOLOGY STACK.....	8
3.1 Frontend.....	8
3.2 Backend.....	9
3.3 Database.....	9
3.4 Cloud Storage.....	9
3.5 Real-Time Communication.....	10
3.6 Authentication and Security.....	10
3.7 Version Control.....	10
CHAPTER 4: SYSTEM ARCHITECTURE.....	11
4.1 High-Level Architecture Diagram.....	11
4.2 Description of Each Layer.....	11
4.3 Deployment Architecture.....	12
CHAPTER 5: DESIGN.....	13
5.1 Database Design Principles.....	13
5.2 ER Diagram / Database Schema.....	13
5.3 Database Tables.....	14
5.4 Design Decisions and Assumptions.....	15
CHAPTER 6: IMPLEMENTATION.....	16
6.1 Module-wise Implementation.....	16
6.2 Front-End Logic.....	19
6.3 API Integration.....	20
6.4 Authentication and Authorization.....	20
6.5 Cloud Storage Integration.....	21
6.6 Real-Time Communication.....	21
6.7 End-to-End Course Learning Workflow.....	21
CHAPTER 7: FEATURES.....	23
7.1 Student Features.....	23
7.2 Instructor Features.....	23
7.3 Administrator Features.....	23
7.4 How Each Feature Works.....	23

CHAPTER 8: TESTING.....	25
8.1 API Testing.....	25
8.2 Integration Testing.....	25
8.3 Tools Used for Testing.....	25
8.4 Test Cases and Results.....	26
CHAPTER 9: DEPLOYMENT.....	27
9.1 Deployment Steps.....	27
9.2 Environment Configuration.....	28
9.3 Hosting of Frontend and Backend.....	28
CHAPTER 10: CHALLENGES AND LIMITATIONS.....	29
10.1 Video Streaming Performance.....	29
10.2 Real-Time Synchronization.....	29
10.3 Role-Based Access.....	29
10.4 Schema Design Trade-offs.....	30
10.5 Current Limitations / Known Issues.....	30
CHAPTER 11: FUTURE ENHANCEMENTS.....	31
11.1 Connecting the Frontend to Live REST APIs.....	31
11.2 Video Upload Pipeline to AWS S3.....	31
11.3 Quiz Engine and Auto-Grading.....	31
11.4 Socket.io Discussion Forum.....	31
11.5 Downloadable PDF Certificates.....	32
11.6 End-to-End Testing.....	32
CHAPTER 12: CONCLUSION.....	32
12.1 Summary of the Project.....	32
12.2 What Was Achieved.....	32
12.3 Skills Learned.....	32
CHAPTER 13: REFERENCES.....	33
CHAPTER 14: APPENDICES.....	34
Appendix A: Database Schema / SQL.....	34
Appendix B: API Reference.....	34
Appendix C: Important Code Snippets.....	34
Appendix D: Installation / Setup Instructions.....	34
Appendix E: Screenshots.....	34
Appendix F: Sample Outputs.....	34

LIST OF FIGURES

Figure 4.1: High-Level System Architecture of the Online Learning Management System.....	11
Figure 5.1: Conceptual ER Diagram of the LMS Entity Groups.....	14
Figure 6.1: LMS User Workflow by Role.....	16
Figure 6.2: JWT-Based Authentication and Authorization Flow (Designed).....	20
Figure 6.3: Course Learning Workflow from Authoring to Certification.....	22
Figure 8.1: Planned Testing Workflow for Each Module.....	25

LIST OF TABLES

Table 1.1: Status Labels Used in This Report.....	2
Table 1.2: User Roles and Their Primary Use of the System.....	3
Table 2.1: Recommended Development Hardware Requirements.....	5
Table 2.2: Software Requirements.....	5
Table 2.3: Functional Requirements Derived from the Project Objectives.....	6
Table 3.1: Technology Stack of the Online Learning Management System.....	8
Table 5.1: Database Entities and Their Purpose.....	14
Table 5.2: Design Decisions, Rationale, and Basis.....	15
Table 6.1: Module Status Summary (Review-2 Stage, Approximately 40% Overall Progress).....	16
Table 6.2: User and Role Management Module Summary.....	17
Table 6.3: Course Builder Module Summary.....	17
Table 6.4: Video Lecture Streaming Module Summary.....	18
Table 6.5: Quiz and Assignment Engine Module Summary.....	18
Table 6.6: Progress Tracking and Certificates Module Summary.....	18
Table 6.7: Discussion Forum Module Summary.....	19
Table 6.8: Admin Analytics Module Summary.....	19
Table 8.1: Planned Test Cases and Results.....	26
Table 11.1: Current Status versus Planned Work.....	31

REPORT CONVENTIONS

This report is written from the information available in the project material at the time of writing. Two conventions are used so that the reader can tell verified content from content that still has to be supplied.

Status labels. Every function is described as Implemented, Designed, Planned, or Future, as defined in Table 1.1. The report records that the project was approximately 40% complete at the Review-2 stage and does not claim any module as fully implemented.

Highlighted placeholders. Text shown in highlighted square brackets, such as [DETAIL REQUIRED], marks information that is not available in the supplied material and that must be completed by the team from the project files, for example exact table names, endpoint lists, test results, and screenshots. These placeholders are intentional. They replace guesses, and they must be removed or filled before the report is submitted.

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Formal education has traditionally been organised around the physical classroom: a fixed group of learners, a fixed schedule, and an instructor who delivers material in real time. This model works well for small cohorts, but it imposes structural limits as the number of learners grows. Instructor attention is a finite resource, so the time available to answer individual questions, review individual work, and notice individual difficulties shrinks as class size increases. A learner who misses a session, or who needs to revisit a concept at a different pace, has no guaranteed way of doing so unless the material has been captured and made available somewhere else.

In practice, instructors and institutions compensate by assembling informal collections of tools. Lecture videos are shared through links, notes circulate through e-mail or messaging groups, assignments are collected on separate portals, and marks are maintained in spreadsheets. Each tool solves one part of the problem, but none of them holds a complete picture. A learner must move between several places to find a lecture, submit an assignment, and check a score, while the instructor has no single place in which to see how those activities add up for any one learner.

Three consequences follow from this fragmentation. First, progress is difficult to track: when lectures, quizzes, and assignments live in different systems, an instructor can only estimate how far each learner has advanced, and usually learns about a struggling learner after the difficulty has already affected the result. Second, assessment management becomes a manual task, because question papers, submissions, and scores must be collected and reconciled by hand. Third, certification is weakly supported: a learner who completes a course often holds no single, verifiable record that shows what was completed and when, and an instructor or institution has no consistent rule that connects completion of the course content to the award of a certificate.

Interaction between instructors and learners is also constrained. In a classroom, questions are asked during the session and are rarely preserved for those who were absent or who think of the question later. Outside the classroom, discussion tends to move to informal channels where it is scattered, difficult to search, and disconnected from the course material it refers to.

A Learning Management System (LMS) addresses these problems by treating the complete learning lifecycle (authoring, delivery, assessment, progress tracking, and certification) as one database-backed process instead of a set of disconnected activities. The Online Learning Management System developed in this project is a centralized, web-based platform in which instructors build and publish structured courses, learners study and are assessed within the same environment, and administrators oversee users and platform activity. The remaining chapters of this report describe how that platform was specified, designed, and partially realised, and which parts of it remain to be completed.

1.2 Problem Statement

The limitations described above can be stated as four specific problems, each of which the proposed system is intended to address:

1. **Limited scalability of classroom-only learning.** Delivery that depends on a live session and a single instructor does not extend effectively to a large or distributed group of learners.
2. **Lack of real-time visibility for instructors.** Instructors have no consolidated, current view of learner engagement and progress, and therefore cannot intervene while intervention is still useful.

3. **Rigid tools with limited real-time interaction.** Existing LMS tools may be inflexible in how courses are structured and may offer little support for real-time interaction between participants.
4. **No single verifiable record of learning.** Learners lack one authoritative record that shows what they have completed and that can be tied to certification.

Taken together, these problems define the following system-level problem. Given a population of learners, instructors, and administrators, the system must (a) allow instructors to author and publish structured course content, including video lectures, quizzes, and assignments; (b) deliver that content to enrolled learners at scale; (c) record each learner's activity and progress in a persistent relational store that instructors and administrators can query; (d) issue a certificate when the defined completion criteria are met; and (e) provide a channel for real-time discussion. Access to each of these functions must be restricted according to the role of the authenticated user.

1.3 Scope of the Project

The scope of the project is taken from the finalized abstract and scope definition prepared during the project reviews. It covers the following functional areas, which together form the learning lifecycle described in Section 1.1:

- User and role management, covering the Student, Instructor, and Administrator roles.
- Course authoring through a course builder used by instructors.
- Course delivery, including video lecture streaming from cloud storage.
- Quizzes and assignments as the assessment mechanisms.
- Progress tracking for each learner and the generation of certificates.
- A discussion forum supporting real-time interaction.
- Administrative analytics for platform-level oversight.

The following items are explicitly outside the scope of the project:

- Payment gateway integration and monetary tuition fee transaction processing.
- AI-based continuous remote proctoring through webcam biometric eye-tracking during examination sessions.
- Native standalone mobile applications (iOS and Android native binaries). The system is architected as a fully responsive progressive web application (PWA).
- SCORM / xAPI enterprise legacy package player integration.

Functionality that appears in neither the in-scope nor the out-of-scope list is not claimed by this report.

Because the project was reviewed at an intermediate stage, this report distinguishes four levels of completion for every function it describes. These labels are used consistently in Chapters 6 to 12.

Table 1.1: Status Labels Used in This Report

Label	Meaning
Implemented	Working functionality that has been built and verified. No module is claimed as fully implemented in this report.
Designed	A design artefact exists (architecture, schema, page design, or authentication flow) but the corresponding working feature is not claimed.
Planned	Work that is identified in the project plan and is scheduled to be carried out.
Future	Work beyond the current plan, listed as possible extension.

1.4 Objectives

The project is guided by the following action-oriented objectives. Each is traced to a module or design element in later chapters, and Chapter 12 evaluates how far each has been met.

1. Enable instructors to author and publish structured courses through a course builder.
2. Provide scalable video lecture delivery by storing lecture videos in cloud object storage rather than on the application server.
3. Provide quizzes and assignments so that learner understanding can be assessed within the platform.
4. Track the progress of every enrolled learner and keep that record in the relational database.
5. Support certification, so that a certificate can be generated when a learner satisfies the completion criteria.
6. Support real-time discussion between learners and instructors.
7. Provide administrative analytics that summarise platform usage for administrators.
8. Implement role-based access so that each function is available only to the roles entitled to use it.

1.5 Importance of the Application

The practical value of an LMS lies less in any single feature than in the fact that all learning activity is recorded in one place. When lectures, assessments, progress, and discussion share one data store, questions that are laborious to answer with separate tools become routine queries: which learners have not started a course, how far has a cohort progressed, and which learners have met the requirements for a certificate.

For students, the system provides one location from which a course can be found, studied, assessed, and completed, together with a visible record of progress. Because lecture videos are held in cloud storage and delivered on demand, a learner is not bound to a scheduled session. The progress record and certificate give the learner evidence of completion that does not depend on an instructor's personal records.

For instructors, the platform replaces repeated manual preparation with a structured course builder and gives a current view of learner progress. The instructor can see which parts of a course learners have completed and can respond while the course is still running. Quizzes that are graded by the system, once built, will further reduce the manual effort of assessment.

For administrators, centralizing the learning lifecycle makes the platform manageable. User accounts and roles can be maintained in one place, and aggregate analytics allow the administrator to monitor how the platform is being used without examining individual courses.

1.6 Target Users / Audience

The system is built around three roles. A user holds exactly one role at a time, and the role determines which screens and operations are available. The role model is described technically in Sections 3.6 and 6.4.

Table 1.2: User Roles and Their Primary Use of the System

Role	Primary use of the system
Student	Enrolls in published courses, watches video lectures, attempts quizzes, submits assignments, follows personal progress, obtains certificates, and takes part in discussion.
Instructor	Builds and publishes courses, adds lectures, quizzes, and assignments, monitors the progress of enrolled learners, and takes part in discussion.

Role	Primary use of the system
Administrator	Manages user accounts and roles and reviews analytics that describe platform-level activity.

CHAPTER 2: SYSTEM REQUIREMENTS

Chapter 1 established what the system must do. This chapter states what is needed to build and run it. The hardware and software figures below are recommended development requirements, chosen to be sufficient for the selected technology stack. They are not measured values, and no benchmark of the LMS on specific hardware has been carried out.

2.1 Hardware Requirements

During development, the frontend, the backend server, and the database server can run on one computer. The entire application is built from lightweight components, so ordinary student hardware is adequate. Video lectures are intended to be stored in cloud object storage, which means that the local machine does not need capacity for lecture media.

Table 2.1: Recommended Development Hardware Requirements

Component	Minimum (recommended)	Remark
Processor	Dual-core, 64-bit	A quad-core processor gives faster builds.
Memory (RAM)	4 GB (8 GB preferable)	The browser, Node.js, and MySQL run together.
Free disk space	About 1 GB	Source code, installed packages, and database files.
Network	Internet connection	Required to install packages and, later, to reach cloud storage.
Display	1366 x 768 or higher	Needed to review the responsive interface.

2.2 Software Requirements

The software environment follows from the technology stack selected in the project reviews. Table 2.2 lists the versions used by the team.

Table 2.2: Software Requirements

Software	Version / type	Purpose
Operating system	Windows, Linux, or macOS	Any platform that supports Node.js and MySQL.
Node.js	v20.18.0 (LTS)	Runtime for the backend API server and frontend tooling.
npm	v10.8.2	Installs and manages JavaScript dependencies.
MySQL	MySQL Community Server 8.0.36	Relational database for the LMS data.
Web browser	Google Chrome (v128+) / Mozilla Firefox (v130+)	Runs the React single-page application.
Git	v2.46.0	Version control of the source code.
Code editor	Visual Studio Code (v1.94+) with ESLint, Prettier, and Tailwind CSS IntelliSense	Development environment, code linting, formatting, and styling support.
AWS account	AWS S3 (Free Tier / Academic Cloud Sandbox)	Video lecture storage.

Software	Version / type	Purpose
API testing tool	Postman v11.14 / Thunder Client for VS Code	Used to exercise REST endpoints during testing.

2.3 Development Tools and Frameworks

Each tool in the stack was selected for a specific responsibility, summarised below. A detailed discussion follows in Chapter 3.

- **React.js** builds the user interface as reusable components, which suits an application with several role-specific dashboards that share common elements such as forms and course listings.
- **Node.js with Express.js** hosts the REST API. Node.js allows the backend to be written in the same language as the frontend, and Express.js supplies routing and middleware, which is where authentication and role checks are applied.
- **MySQL** stores users, courses, enrollment and progress data, and assessments in related tables, and provides the integrity guarantees that a record of learning requires.
- **AWS S3** is intended to hold lecture videos, keeping large media files out of the application server and the database.
- **Socket.io** is intended to carry real-time messages for the discussion forum and live notifications.
- **JSON Web Tokens (JWT)** carry the identity and role of an authenticated user between requests.
- **Git and GitHub** record the history of the source code and support collaboration among the three team members. The project is hosted in a GitHub public/academic repository at <https://github.com/team27-kl-lms/lms-core>.

2.4 Functional and Non-functional Requirements

The requirements below are derived directly from the objectives in Section 1.4 and from the architecture in Chapter 4. They are stated here so that the design, features, and testing chapters can be checked against a single list.

Table 2.3: Functional Requirements Derived from the Project Objectives

ID	Requirement	Role
FR-1	The system shall authenticate users and identify each user's role.	All
FR-2	The system shall restrict each operation to the roles entitled to perform it.	All
FR-3	The system shall allow an instructor to create, structure, and publish a course.	Instructor
FR-4	The system shall allow an instructor to add video lectures, quizzes, and assignments to a course.	Instructor
FR-5	The system shall allow a student to enroll in a published course and to access its content.	Student
FR-6	The system shall allow a student to attempt quizzes and submit assignments.	Student
FR-7	The system shall record learner progress and make it visible to the learner and the instructor.	Student, Instructor

ID	Requirement	Role
FR-8	The system shall make a certificate available when completion criteria are met.	Student
FR-9	The system shall provide a discussion forum with real-time updates.	All
FR-10	The system shall present platform-level analytics to administrators and allow user and role management.	Administrator

The non-functional requirements are the qualities expected of the system as a whole. **Data integrity:** related records, such as an enrollment and the course it refers to, must remain consistent. **Security:** protected operations must not be reachable without a valid token of the correct role. **Scalability:** video delivery must not depend on the capacity of the application server, which is the reason for external object storage. **Maintainability:** the three-tier separation and the modular organisation of the code should allow one module to be changed without rewriting the others. **Reliability:** a failed request must not leave partial or inconsistent data in the database.

CHAPTER 3: TECHNOLOGY STACK

This chapter explains the technologies selected for the system and the reasoning behind each choice. The technology stack was finalized during the project reviews, so the descriptions below state what each component contributes to the design. They do not imply that each component is already integrated.

Table 3.1: Technology Stack of the Online Learning Management System

Layer / Concern	Technology	Role in the LMS	Status
Frontend	React.js (single-page application)	User interface, dashboards, forms, course screens	Page designs completed
Backend	Node.js, Express.js	REST API and application logic	Implemented; REST API endpoints operational with Express.js router
Database	MySQL	Relational storage of all LMS data	Schema designed
Cloud storage	AWS S3	Video lecture storage and delivery	Planned
Real-time	Socket.io	Discussion forum, notifications, live updates	Planned
Authentication	JWT, role-based	Identity and role of each request	Flow designed
Version control	Git / GitHub	Source management	Active on GitHub with GitFlow branching workflow

3.1 Frontend

The frontend is a single-page application (SPA) built with React.js. In an SPA, the browser loads the application once and then updates the visible page in response to user actions, retrieving only data from the server instead of complete pages. This produces an interface that responds quickly, which matters in a learning platform where a student moves repeatedly between a course outline, a lecture, and an assessment.

React organises the interface as components: self-contained units that combine layout and behaviour and that can be reused. An LMS has many such repeating elements, including course cards, lecture lists, quiz question forms, progress indicators, and tables of users. Writing each once and reusing it keeps the interface consistent and reduces the amount of code to maintain. Component-based development also fits the three-role structure of the system, because the Student, Instructor, and Administrator dashboards can be assembled from a shared set of components. Client-side navigation moves the user between course, lecture, and dashboard views without a full page reload, and form components collect the inputs for logging in, authoring a course, and submitting assessment answers. The interface is designed to be responsive so that it remains usable on screens of different sizes.

The frontend specification is as follows. **Routing:** react-router-dom v7.x, with PageTransition route animation wrappers. **Build tool:** Vite v6.0, which provides fast Hot Module Replacement (HMR) during development and optimised Rollup production bundles. **Styling:** Tailwind CSS v4.0, used to implement a high-contrast Neo-Brutalist design system with tactile borders and sticker-style drop shadows. **State management:** the React Context API, through AuthContext, LMSContext, and ThemeContext, backed by a safeStorage mechanism that uses session

storage with an in-memory fallback for restricted browser environments such as sandboxed iframes.

3.2 Backend

The backend is a REST API built with Node.js and Express.js. Node.js is a JavaScript runtime that executes code outside the browser, and its event-driven, non-blocking model is well suited to a server that spends much of its time waiting for database queries and network calls. Using JavaScript on both the client and the server also means that the team works in a single language across the full stack.

Express.js is a minimal web framework that provides routing and middleware. Routing associates a URL and an HTTP method with a handler function. Middleware is a chain of functions that each request passes through before reaching its handler, and it is the natural place to implement cross-cutting concerns. In this system, middleware is the intended location for verifying the JSON Web Token and checking the role of the requester before any protected handler runs.

The backend exposes application logic to the frontend through REST, in which resources such as users, courses, and assessments are accessed using standard HTTP methods and exchanged as JSON. The backend is the only component that communicates with the database. This keeps validation and business rules in one place and prevents the browser from having direct access to stored data. The backend source is organised into folders by responsibility, as shown below.

```
server/  
├─ controllers/    # Business logic and request validation  
│                  # (authController, courseController, assessmentController)  
└─ routes/         # [DETAIL REQUIRED: description and remaining folders]
```

[DETAIL REQUIRED: the remainder of the backend folder structure (for example models, middleware, and configuration), which was cut off in the supplied text.]

3.3 Database

MySQL is the relational database management system selected for the project. A relational database stores data in tables of rows and columns and expresses relationships between tables through keys. This suits an LMS, whose data consists of clearly related entities: users who enroll in courses, courses that contain lectures and assessments, and progress records that tie a learner to a course.

According to the project material, the database holds the data for users, courses, enrollment and progress, assessments, and related LMS entities. MySQL contributes transactional behaviour, so that an operation touching several tables either completes fully or not at all, and it supports declared constraints that protect the consistency of the data. The detailed table structure is presented in Chapter 5. The exact table names and columns are not reproduced from memory in this chapter, because they must match the final schema. [DETAIL REQUIRED: final table list and the MySQL driver or access library used.]

3.4 Cloud Storage

Amazon Simple Storage Service (AWS S3) is the cloud object storage service selected for video lectures. Video files are large, and storing them in a relational database or on the application server would consume capacity quickly and make streaming performance depend on that server. Object storage is designed for exactly this kind of data: each file is stored as an object in a bucket and can be retrieved over HTTP.

In the intended design, an instructor uploads a lecture video, the file is placed in S3, and the database stores only the information needed to locate it. A student viewing the lecture receives the video from S3 rather than from the API server. This separation is what the project means by

scalable video delivery. The video upload pipeline has not yet been built and is a planned item (Chapter 11). Access to stored videos is designed to use AWS S3 pre-signed URLs. [DETAIL REQUIRED: bucket configuration and video formats.]

3.5 Real-Time Communication

Socket.io is a library that provides event-based, bidirectional communication between a browser and a server. Unlike ordinary request-response HTTP, where the client must ask before the server can reply, a Socket.io connection stays open, so the server can push a message to connected clients as soon as something happens. The library uses WebSocket where available and handles reconnection.

In this project Socket.io is intended to support three behaviours: the discussion forum, so that a posted message appears for other participants without refreshing the page; live notifications, to inform a user of relevant events; and real-time updates of shared state. Its integration is planned. The challenge of keeping real-time messages consistent with the persistent database record is discussed in Chapter 10.

3.6 Authentication and Security

Access to the system is controlled by JSON Web Tokens (JWT) combined with role-based authorization. A JWT is a compact, signed token consisting of a header, a payload of claims, and a signature. After a user logs in, the server issues a token that identifies the user and the user's role. The client presents the token with later requests, and the server verifies the signature and reads the role claim to decide whether the request may proceed. Because the token carries the information needed to make this decision, the server does not need to keep a session record for each logged-in user.

Three roles are defined. The **Student** role grants access to learning functions: enrollment, content, assessments, progress, certificates, and discussion. The **Instructor** role grants access to authoring and monitoring functions: the course builder, lecture upload, creation of quizzes and assignments, and viewing learner progress. The **Administrator** role grants access to user and role management and to platform analytics. Designing the permission rules that map these three roles to API operations was identified as one of the project's challenges (Section 10.3).

The JWT authentication flow has been designed. The supplied material does not state how passwords are stored or which signing algorithm is used, so no particular hashing or encryption mechanism is claimed in this report. [DETAIL REQUIRED: password storage method, JWT signing algorithm, and token expiry policy.]

3.7 Version Control

Git is a distributed version control system that records every change to the source code as a commit, allowing earlier states to be recovered and different lines of work to be merged. For a team of three, this makes it possible to work on different modules at the same time without overwriting each other's files. GitHub hosts Git repositories remotely and provides a shared reference copy of the project. The team follows a GitFlow branching workflow in a GitHub public/academic repository (<https://github.com/team27-kl-lms/lms-core>).

CHAPTER 4: SYSTEM ARCHITECTURE

Chapter 3 introduced the individual technologies. This chapter shows how they are arranged. The system follows a three-tier architecture in which the user interface, the application logic, and the persistent data are separated into distinct layers, with supporting services attached to the application layer.

4.1 High-Level Architecture Diagram

Figure 4.1 presents the architecture. A React.js single-page application forms the client. It communicates over HTTP with a Node.js and Express.js API server, which in turn reads from and writes to a MySQL database. Three supporting services complete the design: JWT for role-based authentication, AWS S3 for video lecture storage, and Socket.io for real-time discussion.

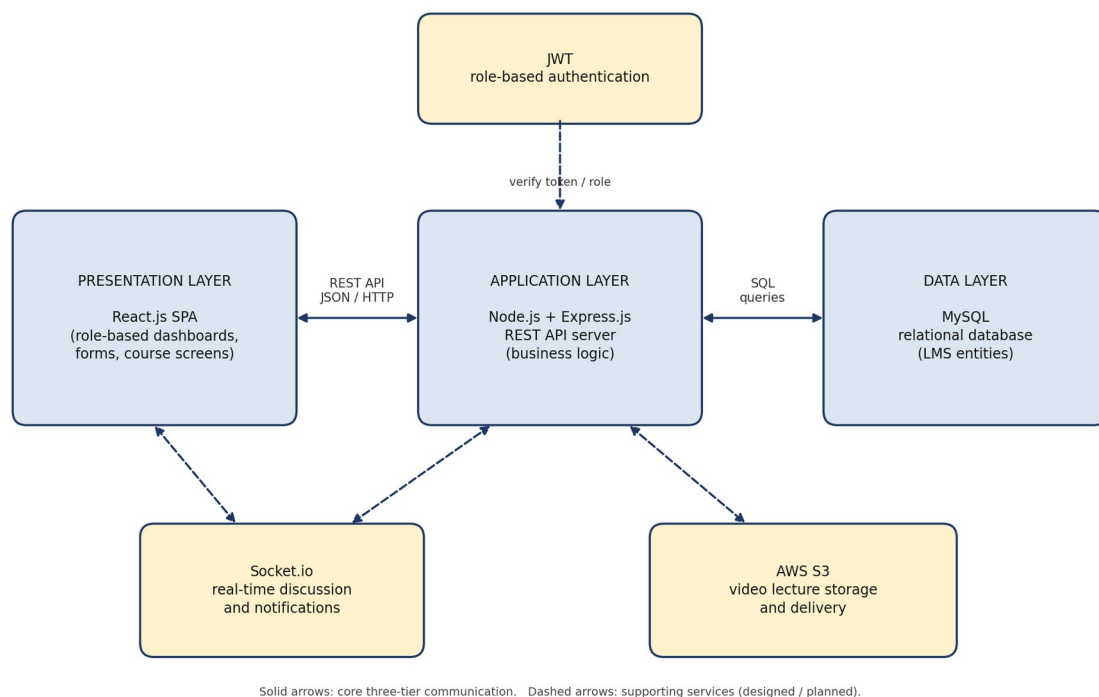


Figure 4.1: High-Level System Architecture of the Online Learning Management System

The architecture was finalized during the project reviews. The figure therefore represents the designed architecture; the extent to which each connection has been built is stated in Chapter 6.

4.2 Description of Each Layer

4.2.1 Presentation Layer

The presentation layer is the part of the system that the user sees and interacts with. It consists of the React.js single-page application running in the user's browser, and its components include the login and registration forms, the role-specific dashboards, the course builder screens used by instructors, the course and lecture views used by students, and the discussion interface. Its responsibility is to display data and to collect user input. It contains no business rules about who may do what, and it never accesses the database directly. The presentation layer communicates with the application layer by sending HTTP requests to the REST API and receiving JSON responses, and it receives real-time messages through a Socket.io connection.

4.2.2 Application Layer

The application layer is the Node.js and Express.js API server. Its components are the route definitions, the middleware that handles authentication and role checks, and the handlers that implement the logic of each LMS module. Its responsibility is to receive requests from the presentation layer, verify the identity and role of the requester, validate the input, apply the rules of the system, and then read or write data in the data layer. It is also the point at which the supporting services are coordinated: it verifies JWTs, mediates video upload to cloud storage, and hosts the Socket.io server. The application layer communicates with the presentation layer through the REST API and Socket.io, with the data layer through SQL queries, and with AWS S3 through the storage service interface.

4.2.3 Data Layer

The data layer is the MySQL relational database. It holds the persistent data of the LMS: user accounts and roles, courses, enrollment and progress records, assessments, and related entities. Its responsibility is to store data reliably and to protect its consistency through keys and constraints. The data layer is accessed only by the application layer, which sends queries and receives result sets. Large video files are deliberately not kept here; the database stores only the information needed to refer to them.

4.2.4 Supporting Services

Three services extend the three layers. **JWT-based authentication** spans the client and the server: the token is issued by the server, held by the client, and verified by the server on each protected request. **AWS S3** is an external service attached to the application layer and holds lecture videos. **Socket.io** is a persistent channel between the client and the server for real-time events. These services are marked as designed or planned in Table 3.1, and none is described here as operational.

4.3 Deployment Architecture

Current status. The system has not been deployed to a hosting environment. No public URL, hosted server, or cloud database exists for this project at the stage covered by this report, and none is claimed.

Intended deployment architecture. The three-tier separation allows each tier to be deployed independently. The intended arrangement is as follows: the React application is built into static files that can be served by a web host or content delivery service; the Node.js and Express.js server runs on an application host; MySQL runs either on a managed database service or on a dedicated server reachable only by the application server; and lecture videos reside in an S3 bucket. Keeping the database unreachable from the public internet, with access only from the API server, follows from the architecture's rule that the data layer is accessed solely by the application layer. This arrangement is a design intention rather than an existing configuration. [DETAIL REQUIRED: chosen hosting providers, if decided.]

CHAPTER 5: DESIGN

The architecture defines where data is stored. This chapter defines how it is organised. The database schema was designed before implementation, in line with the incremental approach of the project, and this chapter documents that design. The detailed schema file was not part of the material available for this report, so wherever a statement would depend on exact table or column names, it is marked for completion rather than assumed.

5.1 Database Design Principles

The schema design follows the standard principles of relational modelling. **Normalization:** each fact should be stored in exactly one place, so that a learner's name, a course title, or a lecture's description is held in a single row and referred to elsewhere by key instead of being copied. This avoids update anomalies, where one copy of a value is changed and another is not. **Entity separation:** users, courses, enrollments, and assessments are distinct real-world things and are therefore modelled as separate entities rather than combined into wide tables.

Primary keys give every row a unique identity. **Foreign keys** express relationships between entities and allow the database to reject a record that refers to something that does not exist, for example an enrollment that points to a missing course. This protects referential integrity. **Validation** is applied at the API layer to check input before it reaches the database, and, where the schema declares them, constraints provide a second line of defence. The official project abstract states that the schema is normalized up to Boyce-Codd Normal Form (BCNF). Together these measures serve data consistency, which is central to a system whose purpose is to keep a trustworthy record of learning.

The Review-2 material identifies a trade-off between normalization and query performance as a schema design challenge (Section 10.4). A fully normalized schema requires joins across several tables to answer questions such as a learner's progress across a course, and the cost of such joins must be weighed against the benefits of avoiding duplicated data. [DETAIL REQUIRED: the specific constraints (UNIQUE, CHECK, NOT NULL) declared in the final schema.]

5.2 ER Diagram / Database Schema

Figure 5.1 shows the conceptual entity model. It is limited to the entity groups that the project material names: users, courses, enrollment and progress data, and assessments. It is deliberately conceptual: attributes and keys are not drawn, because the final schema was not available for transcription, and no table has been added to make the diagram appear more complete.

Conceptual entity model - attributes and keys to be confirmed against the final schema

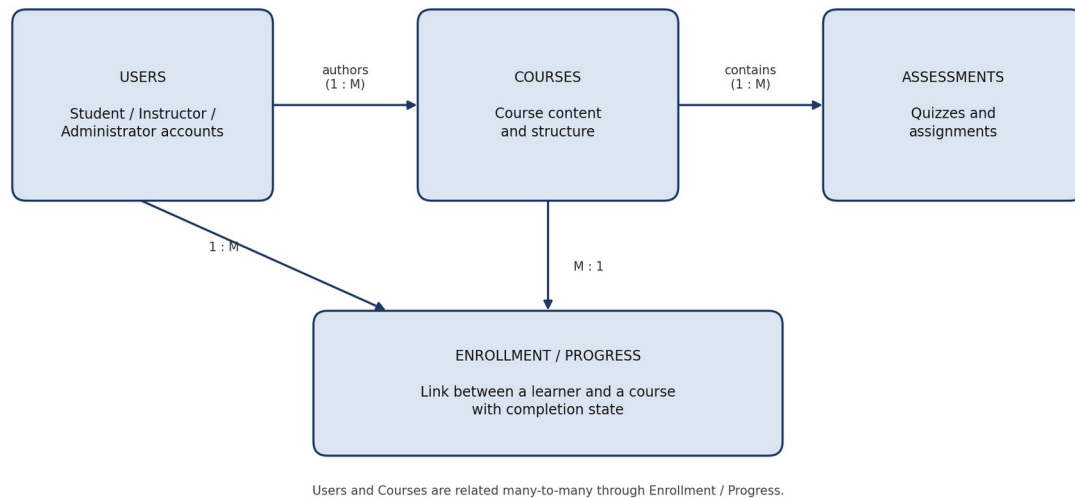


Figure 5.1: Conceptual ER Diagram of the LMS Entity Groups

Users. This entity represents every person who holds an account. It carries the role (Student, Instructor, or Administrator) that drives authorization. It is the parent of courses, through authorship by instructors, and of enrollment records, through participation by students. Its primary key and attributes are [DETAIL REQUIRED].

Courses. This entity represents a course created by an instructor, together with the structure of its content. Each course is created by one user acting as an instructor, and a course can contain many lectures and assessments. The foreign key that identifies the authoring instructor is expected to refer to the users entity. [DETAIL REQUIRED: course attributes, and whether lectures and course sections are separate tables.]

Enrollment and progress. A student can enroll in many courses and a course can have many students, which is a many-to-many relationship. Relational design resolves such a relationship through a linking entity, here the enrollment and progress data, which holds a reference to the user and a reference to the course along with the completion state of that learner in that course. [DETAIL REQUIRED: whether progress is stored per course or per lecture, and the table names.]

Assessments. This entity group represents quizzes and assignments belonging to a course. Each assessment belongs to one course, and a course can have many. [DETAIL REQUIRED: question, answer, submission, and result tables, if they exist as separate entities.]

5.3 Database Tables

Table 5.1 summarises the entity groups in the format required for the schema. Cells that depend on the final schema are marked and must be completed from the SQL schema file before submission.

Table 5.1: Database Entities and Their Purpose

Table name	Primary key	Important columns	Purpose
Users [DETAIL REQUIRED: exact name]	[DETAIL REQUIRED]	Role, identity, credential data [DETAIL REQUIRED]	Stores accounts and the role of each user.

Table name	Primary key	Important columns	Purpose
Courses [DETAIL REQUIRED: exact name]	[DETAIL REQUIRED]	Authoring instructor (foreign key to users) [DETAIL REQUIRED]	Stores courses created by instructors.
Enrollment / progress [DETAIL REQUIRED: exact name]	[DETAIL REQUIRED]	References to user and course; completion state [DETAIL REQUIRED]	Links learners to courses and records progress.
Assessments [DETAIL REQUIRED: exact name(s)]	[DETAIL REQUIRED]	Reference to course; assessment type [DETAIL REQUIRED]	Stores quizzes and assignments.
Other LMS entities [DETAIL REQUIRED]	[DETAIL REQUIRED]	[DETAIL REQUIRED]	Lectures, discussion, and certificate data, if modelled as tables.

5.4 Design Decisions and Assumptions

Table 5.2 lists the main design decisions. The column "Basis" separates decisions that are documented in the project reviews from design reasoning that this report supplies in support of them. The rationale given for every row is reasoned explanation, not measured evidence.

Table 5.2: Design Decisions, Rationale, and Basis

Decision	Rationale	Basis
Relational storage (MySQL)	LMS data consists of clearly related entities, and a record of learning requires consistency across them.	Documented choice; rationale reasoned
Three user roles	Learners, authors, and operators need different capabilities; separating roles limits what any account can do.	Documented
Separate API layer (Express.js)	Keeps validation and rules in one place and keeps the database unreachable from the browser.	Documented architecture; rationale reasoned
Video in cloud object storage (S3)	Large media files should not burden the database or application server, and delivery should scale independently.	Documented choice; rationale reasoned
JWT for authentication	A signed token carrying the role allows each request to be authorised without a stored session.	Documented flow; rationale reasoned
Socket.io for real-time communication	A persistent connection lets the server push discussion messages and notifications.	Documented choice; rationale reasoned
Enrollment as a linking entity	Resolves the many-to-many relationship between users and courses.	Assumption following relational practice; confirm against schema

CHAPTER 6: IMPLEMENTATION

The design in Chapters 4 and 5 is realised module by module. This chapter describes each module, the frontend and API structure, and the authentication, storage, and real-time mechanisms. It also states the status of each part. The Review-2 material records overall progress of approximately 40%: the abstract, scope, architecture, technology stack, database schema, frontend page designs, and JWT authentication flow were completed as design work, while the integration of the live REST APIs, video pipeline, quiz engine, discussion forum, and certificates remained as future work at that stage. Accordingly, the module descriptions below state what each module is designed to do, and the status line of each table states how far it has progressed.

6.1 Module-wise Implementation

Figure 6.1 shows how the three roles move through the system and how their activities connect: courses published by instructors become available to students, and learner activity feeds the analytics seen by administrators.

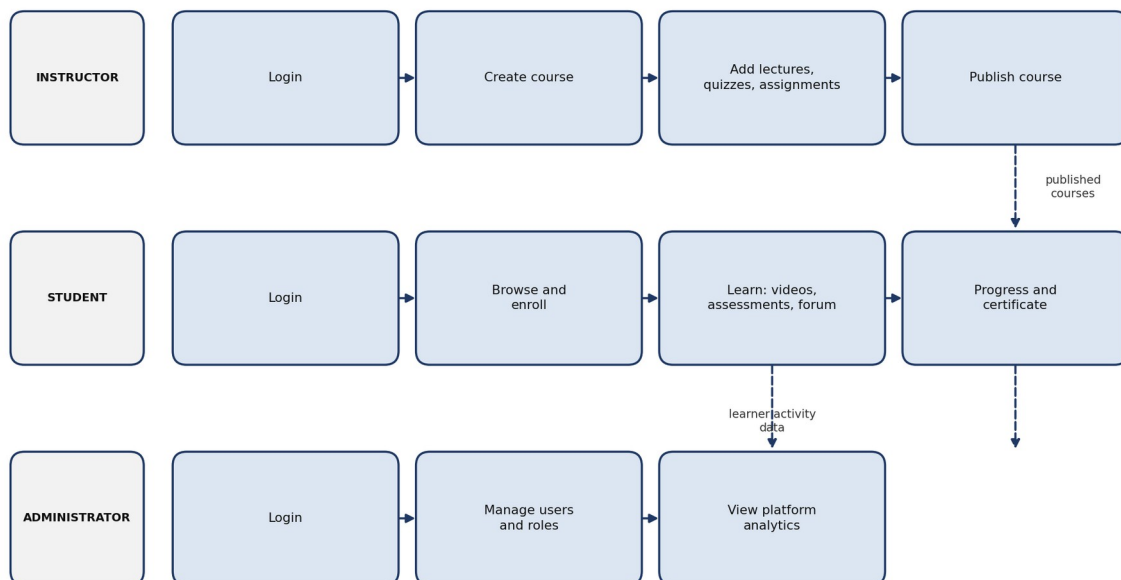


Figure 6.1: LMS User Workflow by Role

The system consists of seven modules. Table 6.1 summarises their status as recorded in the project material.

Table 6.1: Module Status Summary (Review-2 Stage, Approximately 40% Overall Progress)

No.	Module	Primary role	Status
1	User and Role Management	All roles	JWT flow designed; schema designed; working implementation DETAIL REQUIRED
2	Course Builder	Instructor	Page designs completed; API connection planned

No.	Module	Primary role	Status
3	Video Lecture Streaming	Instructor, Student	Architecture finalized; upload pipeline planned
4	Quiz and Assignment Engine	Instructor, Student	Quiz engine and auto-grading planned
5	Progress Tracking and Certificates	Student, Instructor	Schema designed; PDF certificates planned
6	Discussion Forum	All roles	Socket.io integration planned
7	Admin Analytics	Administrator	Page design completed; implementation [DETAIL REQUIRED]

6.1.1 User and Role Management

Table 6.2: User and Role Management Module Summary

Purpose	Create and identify user accounts and assign each a role, so that all other modules can restrict functionality.
Inputs	Registration and login details; role assignments made by an administrator.
Processing	Credentials are checked at login and a JWT carrying the user's role is issued (Section 6.4). Role changes are applied by an administrator.
Data involved	The users entity (Chapter 5).
Output	A token identifying the user and role; user listings for the administrator.
User role	All roles log in; the Administrator manages accounts.
Status	Authentication flow designed. [DETAIL REQUIRED: whether login and registration endpoints are working.]

6.1.2 Course Builder

Table 6.3: Course Builder Module Summary

Purpose	Let instructors create a course, organise its content, and publish it for enrollment.
Inputs	Course details and structure entered through forms; references to lectures and assessments.
Processing	The input is validated, stored against the instructor's account, and marked as published when the instructor releases it.
Data involved	The courses entity and its related content entities.
Output	A stored course visible to students after publication.
User role	Instructor.
Status	Frontend page designs completed. Connection to live REST APIs planned.

6.1.3 Video Lecture Streaming

Table 6.4: Video Lecture Streaming Module Summary

Purpose	Deliver recorded lectures to learners without loading the application server or database.
Inputs	Lecture video files uploaded by instructors.
Processing	In the intended design the file is stored in AWS S3, a reference to it is saved with the lecture record, and playback retrieves the video from S3 (Section 6.5).
Data involved	Lecture records and the S3 object reference. [DETAIL REQUIRED: field names.]
Output	A playable lecture for enrolled students.
User role	Instructor uploads; Student views.
Status	Architecture finalized. The video upload pipeline to S3 is planned and not yet built.

6.1.4 Quiz and Assignment Engine

Table 6.5: Quiz and Assignment Engine Module Summary

Purpose	Assess learner understanding through quizzes and assignments.
Inputs	Quiz questions and assignment instructions from the instructor; answers and submissions from the student.
Processing	Quiz answers are intended to be graded automatically against stored correct answers. The handling and evaluation of assignment submissions is [DETAIL REQUIRED].
Data involved	The assessments entity group and result data. [DETAIL REQUIRED: table names.]
Output	A score and result for each attempt, available to the learner and the instructor.
User role	Instructor creates; Student attempts.
Status	Planned: quiz engine and auto-grading are listed as future work in the Review-2 material.

6.1.5 Progress Tracking and Certificates

Table 6.6: Progress Tracking and Certificates Module Summary

Purpose	Maintain a record of each learner's advancement and issue a certificate on completion.
Inputs	Learner actions such as completing lectures and assessments.
Processing	Completion events update the learner's progress record. When the completion criteria are satisfied, a certificate becomes available. [DETAIL REQUIRED: the exact completion criteria.]
Data involved	The enrollment and progress entity.
Output	Progress indicators for learners and instructors; a downloadable PDF certificate.
User role	Student views; Instructor monitors.
Status	Schema designed. PDF certificate generation is planned.

6.1.6 Discussion Forum

Table 6.7: Discussion Forum Module Summary

Purpose	Provide a shared space for questions and discussion between learners and instructors.
Inputs	Messages posted by users.
Processing	Messages are stored and pushed to connected participants through Socket.io (Section 6.6).
Data involved	Discussion data. [DETAIL REQUIRED: table names.]
Output	Messages displayed to participants as they are posted.
User role	Student and Instructor.
Status	Planned: Socket.io integration listed as future work.

6.1.7 Admin Analytics

Table 6.8: Admin Analytics Module Summary

Purpose	Give administrators an aggregate view of platform activity.
Inputs	Data already recorded by the other modules.
Processing	The data layer is queried and the results are summarised for display. [DETAIL REQUIRED: the list of metrics shown.]
Data involved	Users, courses, enrollment, and progress data.
Output	A dashboard of platform-level figures.
User role	Administrator.
Status	Frontend design completed. [DETAIL REQUIRED: implementation status of the queries.]

6.2 Front-End Logic

The frontend design was completed as a set of page designs for the React application. The interface is organised by role. Public screens, such as login and registration, are available before authentication. After login, each user is directed to a dashboard appropriate to the role: the student area for courses, lectures, assessments, and progress; the instructor area for the course builder and learner monitoring; and the administrator area for user management and analytics. The discussion interface is shared between roles.

Within these screens, forms collect input for login, course authoring, and assessment answers; dashboards present summaries of courses, progress, and analytics; and course interfaces present the course outline and the lecture view. Reusable components, such as lists, cards, and form fields, are shared among the role areas, which keeps the three interfaces consistent. Navigation between screens occurs on the client without reloading the page, and the screens that depend on the logged-in user need access to the user's identity and role. Shared state, such as the logged-in user, the LMS data, and the display theme, is held in React Context providers (AuthContext, LMSContext, and ThemeContext). [DETAIL REQUIRED: list of implemented pages and component structure.]

At the Review-2 stage the screens existed as completed designs, and connecting them to live REST APIs was identified as the next step. The frontend is therefore described as designed, not as fully functional.

6.3 API Integration

The React frontend communicates with the Node.js and Express.js backend over HTTP using a REST interface with JSON payloads. The frontend sends a request to a URL on the API server, the server processes it and returns a status code and a JSON body, and the frontend updates the screen from the response. A request to a protected resource also carries the user's JWT, which the backend verifies before processing (Section 6.4). All data displayed by the frontend originates from such calls, because the browser has no direct connection to MySQL.

This report does not list endpoint names, because none were supplied in the project material, and none have been invented. The planned work item "connect frontend screens to live REST APIs" corresponds to replacing the static content of the designed screens with data retrieved through these calls. [DETAIL REQUIRED: endpoint list, to be completed in Appendix B.]

6.4 Authentication and Authorization

Figure 6.2 shows the authentication flow as designed. At login, the client sends credentials to the API server (step 1). The server queries the database for the user record and role (steps 2 and 3), verifies the credentials, and signs a JWT containing the user's identity and role claim (step 4). The token is returned to the client (step 5). On each subsequent request to a protected resource, the client presents the token (step 6). The server verifies the signature, reads the role claim, and checks whether that role may use the requested route (step 7). If the check passes, the server returns the requested data; otherwise it rejects the request (step 8).

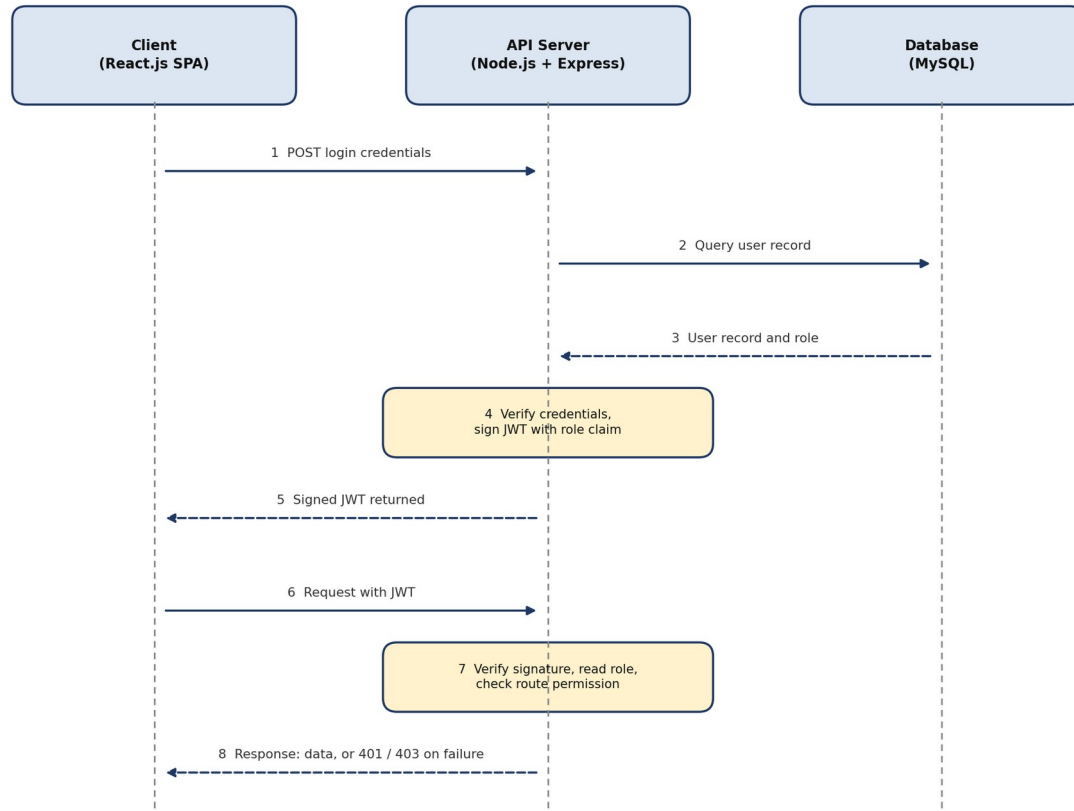


Figure 6.2: JWT-Based Authentication and Authorization Flow (Designed)

Authentication establishes who the user is, and authorization decides what that user may do. The system distinguishes the two: a valid token proves identity, but access to a route depends additionally on the role in the token. A Student token must not be accepted on an instructor-only operation, and neither of those may reach administrator functions. The conventional HTTP responses for these failures are 401 (unauthenticated) and 403 (authenticated but not permitted).

Design versus implementation. The Review-2 material records that the JWT authentication flow was designed. This report therefore presents Figure 6.2 as a design. [DETAIL REQUIRED: whether login, token issue, and the role-checking middleware are coded and working, and how the role of an administrator account is created.]

6.5 Cloud Storage Integration

The planned video workflow has four stages. First, an instructor selects a lecture video in the course builder. Second, the video is transferred to an S3 bucket by the upload pipeline. Third, the application stores a reference to the uploaded object together with the lecture record in MySQL. Fourth, when an enrolled student opens the lecture, the player retrieves the video from S3 using that reference, so the bytes of the video do not pass through the database.

The main performance question for this workflow, identified in the project reviews, is ensuring smooth playback on slower connections (Section 10.1). At the time of this report the upload pipeline is not built, so the workflow is a plan and not an operating feature. [DETAIL REQUIRED: whether upload is performed through the API server or directly from the browser, how access is restricted to enrolled students, and the maximum file size.]

6.6 Real-Time Communication

Socket.io is planned to be attached to the Node.js server so that browser clients keep an open connection to it. When a user posts a discussion message, the message is sent to the server over this connection, the server stores it in the database, and the server then emits it to the other connected participants, whose screens update without a refresh. The same mechanism is intended for live notifications.

Storing the message in addition to broadcasting it matters, because a user who joins later must be able to read the earlier discussion. Maintaining agreement between the broadcast messages and the stored record is the synchronization challenge discussed in Section 10.2. The integration has not been built and remains a planned item. [DETAIL REQUIRED: whether discussion is organised per course, and the notification events to be supported.]

6.7 End-to-End Course Learning Workflow

Figure 6.3 joins the modules into the sequence that the system is designed to support. An instructor creates a course and adds learning content, then publishes it. A student enrolls, accesses the content, and completes assessments. Progress is recorded as the student advances, and a certificate is made available when the completion requirements are met. Section 7.4 explains each step as a feature workflow.

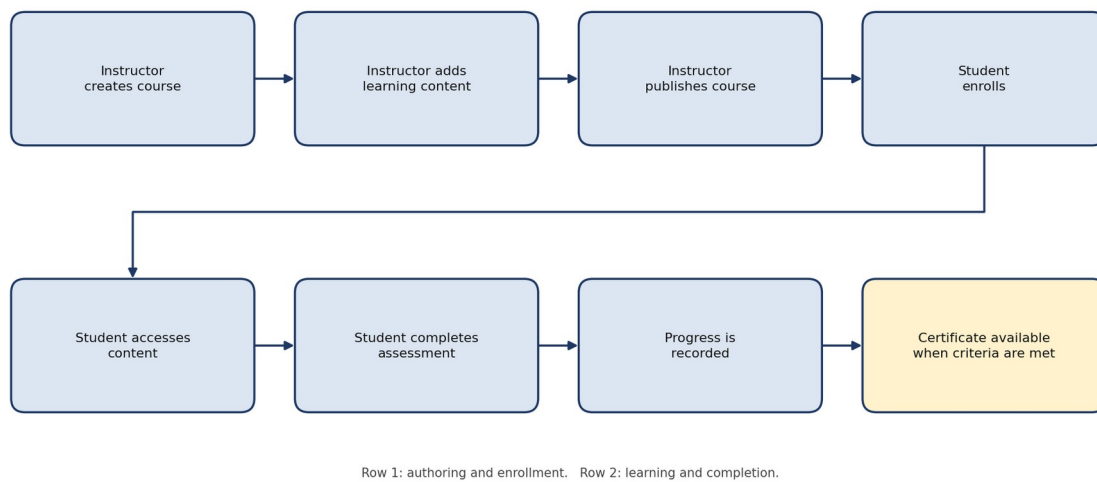


Figure 6.3: Course Learning Workflow from Authoring to Certification

CHAPTER 7: FEATURES

Chapter 6 described the system by module. This chapter describes it from the point of view of each user: what each role can do and how those actions work. The features listed are those defined in the project scope. Feature status follows Table 6.1: all features below are within the designed scope, and none is claimed as fully operational unless Chapter 6 says so.

7.1 Student Features

- **Course enrollment:** a student can browse published courses and enroll in them.
- **Learning:** an enrolled student can view lecture videos streamed from cloud storage and study the course content.
- **Assessments:** a student can attempt quizzes and submit assignments.
- **Progress tracking:** a student can see how much of a course has been completed.
- **Certification:** a student can obtain a certificate after meeting the completion requirements.
- **Discussion:** a student can take part in the course discussion forum.

7.2 Instructor Features

- **Course authoring:** an instructor can create and structure a course using the course builder.
- **Course publishing:** an instructor decides when a course becomes available to students.
- **Lecture upload:** an instructor adds video lectures, which are stored in cloud storage.
- **Quiz creation:** an instructor creates quizzes that can be graded automatically.
- **Assignment creation:** an instructor creates assignments for students to submit.
- **Progress monitoring:** an instructor reviews the progress of learners enrolled in the course.

7.3 Administrator Features

- **User and role management:** an administrator maintains accounts and their roles.
- **Analytics:** an administrator reviews aggregate figures that describe platform activity.
- **Platform monitoring:** the administrator uses these figures to oversee how the platform is used. [DETAIL REQUIRED: monitoring functions beyond analytics, if any.]

7.4 How Each Feature Works

Course creation and publishing. The instructor opens the course builder and enters the course details. The frontend sends the data to the API, which validates it and stores it with a reference to the instructor. The instructor adds lectures, quizzes, and assignments, and then publishes the course. Only after publication is the course visible to students.

Enrollment and access. A student selects a published course and enrolls. The API creates an enrollment record linking the student and the course. Because content access depends on that record, the API can allow the student to open the course content and refuse users who are not enrolled. [DETAIL REQUIRED: whether this access rule is enforced on every content request.]

Assessment and progress. The student attempts a quiz or submits an assignment. The system records the outcome, and the quiz result is determined by comparing the answers with those stored by the instructor once the quiz engine is built. Each completed lecture or assessment updates the learner's progress record, which is the data shown to the student and to the instructor.

Certification. When the progress record shows that the completion criteria are satisfied, the certificate becomes available to the student. Generation of a downloadable PDF certificate is a planned item. [DETAIL REQUIRED: criteria and certificate contents.]

Discussion. A participant posts a message, the server stores it and delivers it to the other connected participants through Socket.io (Section 6.6).

Administration. The administrator opens the analytics view, which summarises data from the other modules, and uses the user management function to maintain accounts and roles.

CHAPTER 8: TESTING

This chapter sets out how the LMS is to be verified. The Review-2 material lists end-to-end testing as outstanding work, and no test results were supplied. For that reason, the chapter describes the testing approach and the planned test cases, and leaves every actual result and status open. No result in this chapter has been invented.

8.1 API Testing

API testing exercises each REST endpoint directly, without the frontend, by sending requests and checking the status code and body of the response. It is the most direct way to verify authentication, role restrictions, and validation, because requests can be made with or without a token and with deliberately invalid data. The tools used for this purpose are Postman (v11.14) and Thunder Client for VS Code. Once the tool is confirmed, each module's endpoints should be grouped, and requests that depend on one another (for example, creating a course and then enrolling in it) should be chained so that a full workflow can be run in sequence.

8.2 Integration Testing

Integration testing verifies that the layers work correctly together. It follows the data path of Figure 4.1: the frontend sends a request to the backend; the backend applies authentication and rules and reads or writes the MySQL database; and the supporting services respond, namely S3 for video access and Socket.io for real-time messages. Testing against a real running database, rather than mocked data, is important for this system, since many of its guarantees (for example, that an enrollment refers to an existing course) are provided by the database. Testing the frontend-to-API connection is the last stage, because it depends on the planned connection of the screens to live APIs. [DETAIL REQUIRED: integration testing actually carried out to date.]

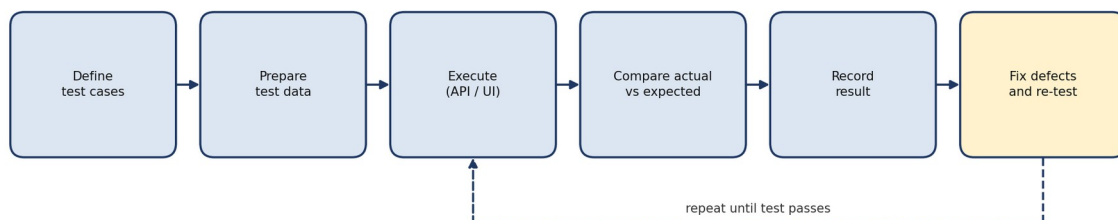


Figure 8.1: Planned Testing Workflow for Each Module

8.3 Tools Used for Testing

The testing approach uses the following tools:

- REST API requests and assertions: Postman v11.14 and Thunder Client for VS Code.
- Direct inspection of database contents: [DETAIL REQUIRED: for example the MySQL command-line client or a graphical client].
- Manual verification of the React interface: a web browser.
- Automated test suite: [DETAIL REQUIRED: if any].

8.4 Test Cases and Results

Table 8.1 lists the planned test cases in the standard format. The cases cover the categories identified for the project. Cases that depend on modules not yet built (video access, discussion, and assessment submission) can be executed only after those modules exist. No actual result has been supplied, so every result and status is left open to be filled from the team's test log.

Table 8.1: Planned Test Cases and Results

Test ID	Test case	Expected result	Actual result	Status
T-01	Login with valid credentials	Login succeeds and a token with the user's role is returned	[ACTUAL RESULT TO BE PROVIDED]	[STATUS TO BE PROVIDED]
T-02	Login with an incorrect password	Login is rejected (401) and no token is issued	[ACTUAL RESULT TO BE PROVIDED]	[STATUS TO BE PROVIDED]
T-03	Access a protected route without a token	Request is rejected (401)	[ACTUAL RESULT TO BE PROVIDED]	[STATUS TO BE PROVIDED]
T-04	Student token used on an instructor-only operation	Request is rejected (403)	[ACTUAL RESULT TO BE PROVIDED]	[STATUS TO BE PROVIDED]
T-05	Instructor token used on an administrator-only operation	Request is rejected (403)	[ACTUAL RESULT TO BE PROVIDED]	[STATUS TO BE PROVIDED]
T-06	Administrator accesses user management	Request is accepted	[ACTUAL RESULT TO BE PROVIDED]	[STATUS TO BE PROVIDED]
T-07	Instructor creates a course with valid data	Course is stored and returned	[ACTUAL RESULT TO BE PROVIDED]	[STATUS TO BE PROVIDED]
T-08	Student enrolls in a published course	Enrollment record is created	[ACTUAL RESULT TO BE PROVIDED]	[STATUS TO BE PROVIDED]
T-09	Student enrolls in the same course twice	Duplicate is handled according to the schema rule [DETAIL REQUIRED]	[ACTUAL RESULT TO BE PROVIDED]	[STATUS TO BE PROVIDED]

Test ID	Test case	Expected result	Actual result	Status
T-10	Student submits an assessment	Submission is stored and result is returned	[ACTUAL RESULT TO BE PROVIDED]	[STATUS TO BE PROVIDED]
T-11	Completion of a lecture or assessment	Progress record is updated	[ACTUAL RESULT TO BE PROVIDED]	[STATUS TO BE PROVIDED]
T-12	Enrolled student requests a lecture video	Video is delivered from cloud storage	[ACTUAL RESULT TO BE PROVIDED]	[STATUS TO BE PROVIDED]
T-13	Participant posts a discussion message	Message is stored and shown to other participants	[ACTUAL RESULT TO BE PROVIDED]	[STATUS TO BE PROVIDED]
T-14	Request with missing or invalid input	Request is rejected with an error message	[ACTUAL RESULT TO BE PROVIDED]	[STATUS TO BE PROVIDED]
T-15	Restart of the server after data entry	Data persists in the database	[ACTUAL RESULT TO BE PROVIDED]	[STATUS TO BE PROVIDED]

CHAPTER 9: DEPLOYMENT

Testing establishes that the system behaves correctly; deployment makes it available to its users. This chapter states what exists at the time of this report and what is intended.

9.1 Deployment Steps

Current status. The project has not been deployed to a hosting environment. Any use of the system at this stage is on development machines.

Local development setup (intended procedure). The steps below follow from the architecture and the selected tools. The folder names and script names are not confirmed, and they should be replaced with the actual ones. [DETAIL REQUIRED: repository layout and script names.]

1. Install Node.js, npm, MySQL, and Git.
2. Clone the project repository.
3. Create the MySQL database and load the schema.
4. Configure the environment variables for the backend (Section 9.2).
5. Install the backend dependencies and start the API server.
6. Install the frontend dependencies and start the React application.
7. Open the application in a browser and log in with an account for each role.

Intended production deployment. The arrangement described in Section 4.3 would be followed: static frontend files on a web host, the API server on an application host, MySQL on a protected database host, and videos in S3. [DETAIL REQUIRED: decided providers and steps.]

9.2 Environment Configuration

Configuration that differs between machines, and values that must remain secret, should be kept in environment variables loaded at start-up, not written into the source code. The environment file itself must be excluded from version control, and a template file that lists the required variables with placeholder values can be committed so that the configuration is documented without exposing secrets. The categories of configuration needed are:

- **Backend configuration:** the port on which the API server listens and the permitted origin of the frontend.
- **Database configuration:** the host, port, user, password, and database name used to connect to MySQL.
- **JWT configuration:** the secret used to sign tokens and the token expiry period.
- **Cloud storage configuration:** the AWS region, bucket name, and access credentials for S3, which apply once the video pipeline is built.

Real values are deliberately not shown in this report. [DETAIL REQUIRED: the actual variable names, to be listed with placeholder values only.]

9.3 Hosting of Frontend and Backend

Neither the frontend nor the backend is hosted at present, and no URL or cloud infrastructure exists for the project. This section is therefore limited to the intended approach: the frontend would be built into static files and served from a web host or content delivery service, and the backend would run as a Node.js process on an application host and connect to a database that is not accessible directly from the internet. [DETAIL REQUIRED: if hosting is arranged before submission, record the provider and configuration here.]

CHAPTER 10: CHALLENGES AND LIMITATIONS

The Review-2 presentation identifies four technical challenges. Each is analysed below in the same order: the problem, its cause, its technical impact, the solution or planned solution, and the current status. The presentation names the challenges but does not record finalized solutions. Where a solution has not been documented, this report says so and gives general engineering approaches that are being considered, which are identified as such and not presented as adopted decisions.

10.1 Video Streaming Performance

Problem. Lecture playback must be smooth when the learner has a slow connection.

Cause. Video files are large, and the rate at which a learner can receive them is limited by the learner's own network, which the system does not control.

Technical impact. If the video is delivered poorly, the learner experiences repeated pauses or failed playback, and the usefulness of the scalable-delivery objective is reduced.

Solution / planned solution. Storing the videos in AWS S3 is the documented design decision that keeps delivery independent of the API server. Measures specific to slow connections have not been recorded. [DETAIL REQUIRED: any additional measures decided.] General approaches include offering several quality levels and delivering video in segments, but they are under consideration only.

Current status. Open. The upload pipeline is not built, so playback performance has not been tested.

10.2 Real-Time Synchronization

Problem. Discussion messages and progress updates must remain consistent for all users who see them.

Cause. Real-time messages travel through Socket.io connections while the authoritative data is held in MySQL. These are two separate paths, and a message can reach one without reaching the other if a connection drops or a database write fails.

Technical impact. A user could see a message that was never stored, or could miss a stored message, and displayed progress could differ from the recorded progress.

Solution / planned solution. The design intention described in Section 6.6 is that the database holds the persistent record and that Socket.io delivers updates. A specific consistency mechanism is [DETAIL REQUIRED]. A commonly used approach is to treat the database as the source of truth and to reload from it when a connection is re-established, but this is not recorded as decided.

Current status. Open. Socket.io integration is planned and not yet built.

10.3 Role-Based Access

Problem. Permissions for the Student, Instructor, and Administrator roles must be expressed through JWT-based checks.

Cause. Every operation in the API must be assigned to the roles that may use it, and the decision must be made on the server from the token, because the client cannot be trusted to restrict itself.

Technical impact. An incorrect or missing check could allow a user to perform an operation reserved for another role, for example a student creating a course or reaching administrator functions. This would affect the integrity of the learning record.

Solution / planned solution. The JWT authentication flow has been designed (Figure 6.2), with the role carried in the token and checked by the server on protected routes. The detailed permission matrix is [DETAIL REQUIRED].

Current status. Designed. Verification through the planned test cases T-03 to T-06 has not yet been carried out.

10.4 Schema Design Trade-offs

Problem. The schema must balance normalization against query performance.

Cause. A highly normalized schema avoids duplicated data but requires joins across several tables, while a less normalized schema answers some queries faster at the cost of redundancy.

Technical impact. Excessive joins could slow queries such as progress summaries and analytics as data grows, while excessive duplication could produce inconsistent records.

Solution / planned solution. The schema was designed by applying normalization principles (Section 5.1). Whether any deliberate denormalization was adopted is [DETAIL REQUIRED].

Current status. Schema designed. Its performance with realistic data volumes has not been measured.

10.5 Current Limitations / Known Issues

The following limitations are documented in, or follow directly from, the Review-2 status of the project:

- Overall progress was approximately 40% at the Review-2 stage; most modules are designed and not yet complete.
- The frontend screens are not yet connected to live REST APIs.
- The video upload pipeline to AWS S3 has not been implemented.
- The quiz engine and auto-grading have not been built.
- The Socket.io discussion forum has not been integrated.
- Downloadable PDF certificates are not yet generated.
- End-to-end testing has not been conducted, and no test results are available.
- The system has not been deployed to a hosting environment.

No software defects are recorded in the supplied material. [DETAIL REQUIRED: known bugs, if any.]

CHAPTER 11: FUTURE ENHANCEMENTS

The remaining work follows the plan in the Review-2 presentation. Table 11.1 separates what exists now from what is planned for each item, and the sections that follow explain what each task involves technically.

Table 11.1: Current Status versus Planned Work

Work item	Current	Planned
Frontend integration	Page designs completed	Connect screens to live REST APIs
Video delivery	Architecture finalized	Upload pipeline to AWS S3
Assessment	Assessments in scope; schema designed	Quiz engine and auto-grading
Discussion	Socket.io selected	Integrate Socket.io discussion forum
Certification	Certificate requirement defined	Generate downloadable PDF certificates
Testing	Test approach defined (Chapter 8)	End-to-end testing

11.1 Connecting the Frontend to Live REST APIs

The designed screens must be linked to the backend so that they display and modify real data. For each screen, this means identifying the API operations it needs, calling them from the React components, handling loading and error states, and attaching the JWT to protected requests. This is the integration step that turns the page designs into a working application, and it is the point at which the role-specific dashboards become usable. It should be carried out module by module, beginning with login, so that each connection can be checked against the real database before the next is attempted.

11.2 Video Upload Pipeline to AWS S3

The pipeline must allow an instructor to send a video from the course builder to an S3 bucket, record its location with the lecture in MySQL, and later serve it to enrolled students. The design decisions still to be made include whether upload passes through the API server or goes directly from the browser, how access to stored videos is limited to authorised learners, and how large files are handled. Playback performance on slow connections (Section 10.1) should be assessed once the pipeline exists.

11.3 Quiz Engine and Auto-Grading

The quiz engine will let instructors define questions and correct answers, present them to students, and compare submitted answers with the stored ones to produce a score automatically. It requires the assessment tables of the schema, API operations for creating and attempting quizzes, and a rule for how results feed the learner's progress record. Auto-grading reduces the manual workload of instructors and makes results available to learners immediately.

11.4 Socket.io Discussion Forum

This task integrates Socket.io with the Node.js server and the frontend so that discussion messages appear for participants without a page refresh, and so that notifications can be pushed. It must be combined with persistent storage of messages, and the consistency issue described in Section 10.2 must be addressed in the design of the message flow.

11.5 Downloadable PDF Certificates

When a learner satisfies the completion criteria, the system should generate a certificate as a PDF file that the learner can download. This requires the completion criteria to be defined precisely, the learner and course data to be assembled into a certificate, and a means of generating the PDF on the server. The certificate supports the objective of giving learners a verifiable record of completion.

11.6 End-to-End Testing

After the above work, the complete system should be tested from the browser through the API and the database to the supporting services, using the test cases in Table 8.1. The results should be recorded in that table, and defects found should be corrected and re-tested. This work also provides the evidence that the objectives of Section 1.4 have been met.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

This project addresses the limits of classroom-only learning by specifying and designing an Online Learning Management System that brings authoring, delivery, assessment, progress tracking, certification, and discussion into one database-backed platform. The system uses a three-tier architecture with a React.js single-page application, a Node.js and Express.js REST API, and a MySQL relational database, supported by AWS S3 for video storage, Socket.io for real-time communication, and JWT for role-based access among Student, Instructor, and Administrator roles. Seven modules were defined, and this report has documented their purpose, intended behaviour, and status.

12.2 What Was Achieved

Based on the Review-2 material, the following have been achieved. The abstract and scope were finalized. The architecture and technology stack were finalized. The database schema was designed. The frontend page designs were completed. The JWT authentication flow was designed. Overall progress was approximately 40% at that review stage.

These results are design and planning results. The integration of live APIs, the video pipeline, the quiz engine, the discussion forum, and the certificates, along with end-to-end testing, remain to be done (Chapter 11), and no test results exist yet. The objectives of Section 1.4 are therefore met at the level of specification and design, and are not yet met at the level of a working system. This report keeps that distinction deliberately, so that it can be updated with implementation and test evidence as the work proceeds.

12.3 Skills Learned

The project provided practice in the following areas:

- **Full-stack development:** organising a web application into a client, a server, and a database, and defining how they communicate.
- **Relational database design:** modelling entities and relationships, resolving a many-to-many relationship, and weighing normalization against query cost.
- **API development:** designing a REST interface with Express.js.
- **Authentication:** understanding how JWT and role claims support authorization.
- **Frontend development:** structuring a React interface into reusable components for several roles.
- **Cloud storage concepts:** understanding why large media is held in object storage.

- **Real-time communication:** understanding event-based messaging with Socket.io and its consistency problem.
- **Testing:** planning API and integration tests with expected results.
- **System architecture:** separating concerns into layers and supporting services.

CHAPTER 13: REFERENCES

[1] React: Official Documentation. <https://react.dev>

[2] Node.js: Official Documentation. <https://nodejs.org/en/docs>

[3] Express.js: Official Documentation. <https://expressjs.com>

[4] MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>

[5] Amazon Simple Storage Service (S3): User Guide. <https://docs.aws.amazon.com/s3/>

[6] Socket.IO: Official Documentation. <https://socket.io/docs/v4/>

[7] Jones, M., Bradley, J., and Sakimura, N. JSON Web Token (JWT). RFC 7519, IETF, 2015. <https://datatracker.ietf.org/doc/html/rfc7519>

[8] jsonwebtoken: npm package documentation.
<https://www.npmjs.com/package/jsonwebtoken>

[9] Git: Official Documentation. <https://git-scm.com/doc>

[10] GitHub Docs. <https://docs.github.com>

[DETAIL REQUIRED: add any further references actually used, such as the database or testing tools confirmed by the team.]

CHAPTER 14: APPENDICES

Only material that exists in the project files can be placed in the appendices. Because the schema, code, endpoint list, screenshots, and outputs were not part of the supplied material, each appendix below is a labelled placeholder that states what should be inserted.

Appendix A: Database Schema / SQL

[DETAIL REQUIRED: insert the SQL schema file (CREATE TABLE statements for every table in Table 5.1). Redraw Figure 5.1 from this schema, adding attributes and keys.]

Appendix B: API Reference

[DETAIL REQUIRED: insert the list of REST endpoints grouped by module, with HTTP method, path, required role, and a sample request and response. No endpoint has been listed here because none was supplied.]

Appendix C: Important Code Snippets

[DETAIL REQUIRED: insert code from the project, for example the JWT verification and role-check middleware and a representative API handler. No code snippet has been written for this report, to avoid presenting code that is not the team's own.]

Appendix D: Installation / Setup Instructions

The general procedure is given in Section 9.1. [DETAIL REQUIRED: replace it with the exact commands, folder names, and environment variable names of the project.]

Appendix E: Screenshots

[DETAIL REQUIRED: insert screenshots of the completed frontend designs and, later, of the running application. Suggested sequence: login page, student dashboard, course page, course builder, quiz page, discussion forum, instructor dashboard, administrator analytics.]

Appendix F: Sample Outputs

[DETAIL REQUIRED: insert sample API responses and, when available, a sample generated certificate.]